

MICROSERVICES COURSE

Session 1



Architecture & Spring Cloud

Service Discovery · Centralized Config · Domain-Driven Design



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Wednesday

10:00 AM – 3:00 PM

Offline Anchor Day

5 Hours

Phase 1

Session 1 of 29

Phase 1 — Foundation & Core Patterns



You are here — Session 1 of 8 in Phase 1. Today we lay the foundation everything else depends on.

Full 29-session roadmap covered in the Program Kickoff session.

Why Not Just Build One Big Application?

THE MONOLITH



One codebase. One deployment.
One database. One team
touches everything.

- ✗ Any change requires a full redeploy
- ✗ One bug can crash the entire system
- ✗ Scaling means scaling EVERYTHING
- ✗ Teams block each other constantly

THE QUESTION

*What if Product, Order, and
Payment could each be built,
deployed, and scaled
INDEPENDENTLY?*

This is the Microservices Architecture —
and it's what we build for the next 29 sessions.

Microservices: Independent, Deployable Services



An architectural style where an application is composed of small, independently deployable services, each owning its own data and business capability.



Gain

- Independent deployment
- Selective scaling
- Team autonomy
- Technology flexibility



Cost

- Operational complexity
- Network latency between services
- Distributed debugging
- Data consistency challenges

Bounded Context — Not Database Tables

A Bounded Context is a logical boundary within which a domain model is defined and consistent. It is the foundation for how we split our platform into services.



Product Catalogue

Read-heavy. Changes rarely. Owned by Catalogue team.



Inventory

Write-heavy. Changes on every order. Owned by Fulfilment team.



Order Management

Orchestrates the purchase flow. Owned by Commerce team.

Different bounded contexts ⇒ different services. Same database table ⇒ NOT a reason to merge services.

How Does Order Service Find Product Service?



Without Service Discovery

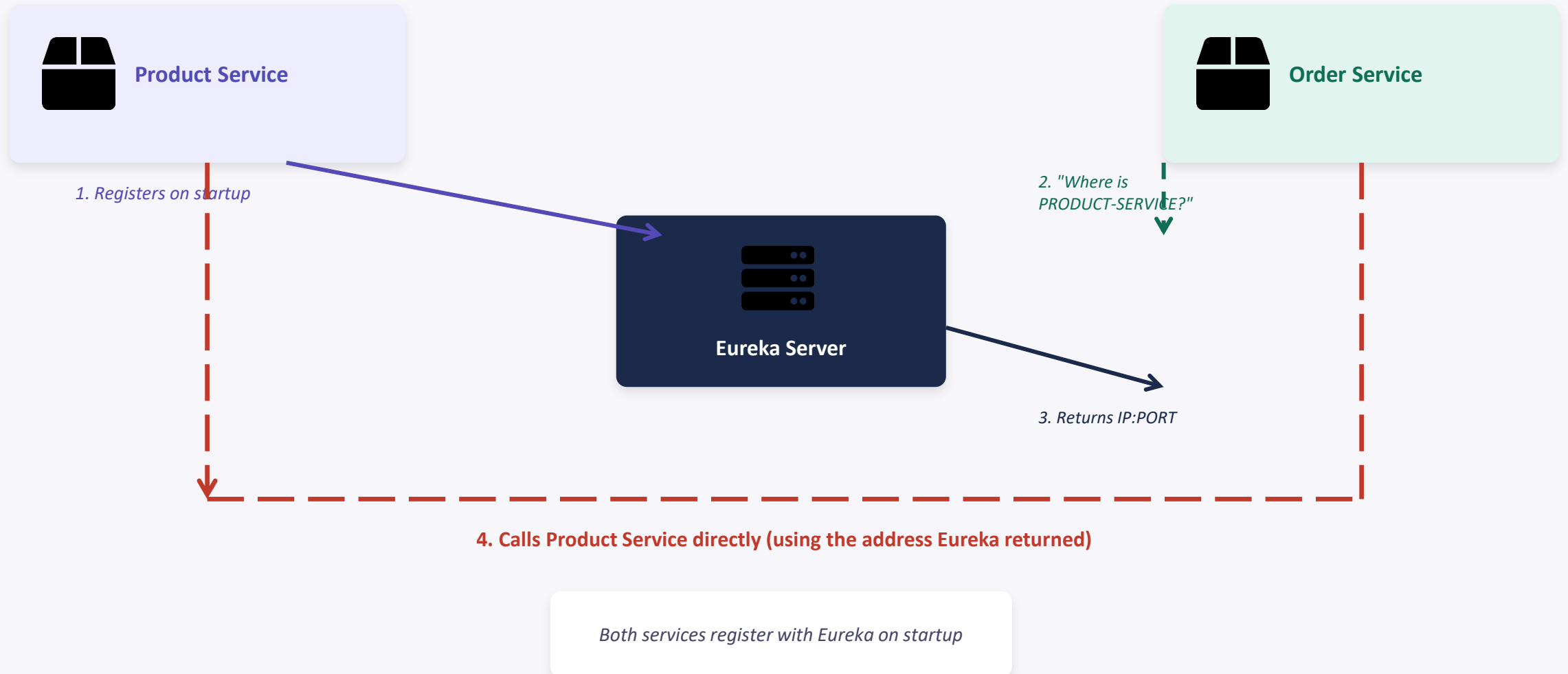
- Hardcoded IP: 10.0.0.42:8081
- Kubernetes reassigns IPs constantly
- Every new instance breaks all callers
- Manual config update on every deploy



With Eureka Service Discovery

- Services register themselves by NAME
- Order Service asks: "Where is PRODUCT-SERVICE?"
- Eureka returns the current live address
- New instances register automatically

How Eureka Service Discovery Works



Setting Up the Eureka Server

`pom.xml` — Eureka Server Dependency

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

`EurekaServerApplication.java` — `@EnableEurekaServer`

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Eureka Server Configuration

`application.yml` — Eureka Server

```
server:
  port: 8761

eureka:
  client:
    register-with-eureka: false    # this IS the registry
    fetch-registry: false
  server:
    enable-self-preservation: false # DEV ONLY
```



Dev vs Production: `enable-self-preservation:false` is fine for a single-node dev setup. In production with multiple Eureka nodes, this must be true to handle network partitions correctly.

What Happens When 8 Services Need the Same Config Change?



Without Centralized Config

- DB password changes? Edit 8 application.yml files
- Redeploy all 8 services to apply the change
- Easy to miss one service — inconsistent state
- Secrets scattered across many repos



With Spring Cloud Config Server

- One Git repo holds config for ALL services
- Change once — services pull the update
- Per-environment config: dev / staging / prod
- Directly implements 12-Factor App, Factor III

The 12-Factor App — Factor III: Config

“Store config in the environment”

Configuration that varies between deployments (database URLs, credentials, feature flags) must NOT be hardcoded in the codebase. It must be externalized.

Codebase

One codebase, tracked in Git, many deploys

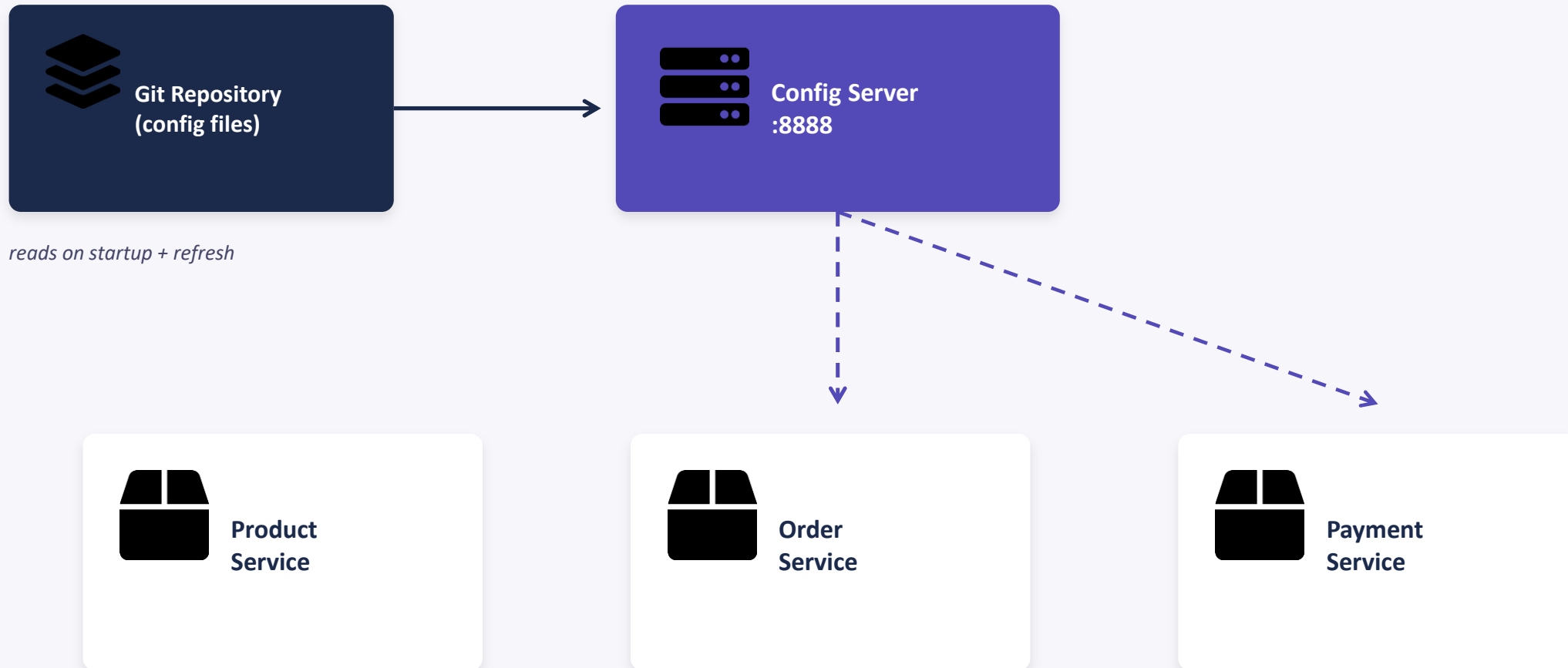
Config

Strict separation of config from code

Backing Services

Treat DB, cache, queue as attached resources

How Spring Cloud Config Server Works



Each service calls Config Server at startup: `spring.config.import: configserver:http://localhost:8888`

Setting Up the Config Server

ConfigServerApplication.java

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

application.yml – Point to Git Repo

```
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/your-org/config-repo
```

Product Service: Connecting to Eureka + Config

`application.yml` — Product Service

```
spring:
  application:
    name: product-service
  config:
    import: "optional:configserver:http://localhost:8888"
    # optional: prefix = service still starts if Config Server is down

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

Verify: Product Service Registers with Eureka

EUREKA DASHBOARD — <http://localhost:8761>

Application	Status	Instances
PRODUCT-SERVICE	UP (1)	localhost:8081

If you see PRODUCT-SERVICE with status UP — Eureka registration is working correctly.

```
Verify Config Server: curl http://localhost:8081/actuator/health → {"status":"UP"}
```

Two Repositories — Platform vs Course Materials



`microservices-pro-platform`

Your working repository. Branch: main. Every session ends with a commit here.



`microservices-pro-course`

Instructor materials. Not for trainee use — session packs, rubrics, slides.

Branch Strategy

`main`

Your trainee working branch

`reference`

Instructor's canonical implementation — compare when stuck

Lab 1 — Build Product Service + Connect Infrastructure

1 Create Eureka Server

Port 8761, @EnableEurekaServer

2 Create Config Server

Port 8888, connect to Git config repo

3 Build Product Service

CRUD REST API + connect to Eureka & Config

4 Write Unit Tests

Minimum 3 tests — service layer logic

Lab 1 Acceptance Criteria

- ✓ Eureka Dashboard (localhost:8761) shows PRODUCT-SERVICE as UP
- ✓ GET /actuator/health on Product Service returns {"status":"UP"}
- ✓ Config Server successfully serves config to Product Service on startup
- ✓ POST /api/products creates a new product and returns 201 Created
- ✓ GET /api/products/{id} returns the correct product or 404
- ✓ All unit tests pass: mvn test
- ✓ Commit pushed: session-01: add-product-service-eureka-config

What We Added to the Platform Today



Capability Added

Service Discovery & Centralized Configuration

Any service can now register itself and discover others by name. Configuration lives in one place, not scattered across 8 services.



Services Updated

NEW: Eureka Server (8761)

NEW: Config Server (8888)

NEW: Product Service (8081)

→ CRUD API + Eureka + Config integration

Common Issues & Solutions



Service not appearing in Eureka

Check `eureka.client.service-url.defaultZone` matches the server's actual port



"Connection refused" to Config Server

Config Server must start BEFORE other services — start it first



optional: prefix missing causes startup failure

Without `optional:`, services fail to start if Config Server is down



Eureka shows service as DOWN despite running

Wait 30s — Eureka heartbeat interval; this is expected, not a bug



Port already in use (8761/8888/8081)

Run: `lsof -i :8761` then kill the conflicting process



Daily Quiz

8 Questions · 10 Minutes

Topics: Service Discovery · Bounded Context · 12-Factor App · Eureka · Spring Cloud Config

Checkpoint Commit & Homework

Checkpoint — Definition of Done

- ☐ Eureka shows Product Service UP
- ☐ Config Server serves config correctly
- ☐ POST/GET /api/products works
- ☐ All unit tests pass
- ☐ Pushed: session-01: ...

Pre-Session 2 Reading (15 min)

Spring Cloud Gateway — Overview
spring.io/projects/spring-cloud-gateway

Cold Call Questions for Session 2:

- What problem does an API Gateway solve?
- What is a Route Predicate?
- Why use lb:// instead of http://?

Phase 1 Roadmap — Sessions 2 through 8



Today we laid the foundation. Every session from here adds one real capability to the platform.

What You Should Remember



Bounded Context, not tables

Services are split by business domain ownership — not by database structure



Eureka removes hardcoded addresses

Services register by name and discover each other dynamically



Config Server centralizes settings

One source of truth, implementing 12-Factor App Factor III



This is the foundation

Every pattern from Session 2 onward builds directly on what you built today



Session 1 Complete

Foundation laid: Service Discovery + Centralized Configuration

Next: Session 2 — API Gateway Core · Monday, 3:00–5:30 PM, Online

microservices-pro-course · microservices-pro-platform · Dr. Sayed Baladoh